

System Design: From Requirements to Reality

A Complete Guide for JHUB Africa Students — From User Needs to a Deployable System

Presented by Dr. Lawrence Nderu

JHUB AFRICA / JKUAT

JHUB Africa Attachment Programme — July 2026

| Training Overview and Objectives

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

“

"A great idea without a great design is just a wish. System design is the discipline that turns your requirement document into a blueprint the whole team can build from."

The Core Discipline

System design bridges the critical gap between **WHAT** a system must do and **HOW** it will actually be implemented. It provides the technical clarity needed to:

- ✓ Ensure consistency across the development team.
- ✓ Prevent expensive rework in resource-limited settings.
- 💖 Demonstrate technical maturity to JHUB partners.

WHAT YOU WILL MASTER



REQUIREMENT EXTRACTION

Identify actors, functional needs, and constraints from raw SRS documents.



STRUCTURAL BLUEPRINTS

Design Information Architecture and User Flows for seamless interaction.



DATA INTEGRITY

Construct robust ERDs and apply normalisation to ensure data quality.



SYSTEM ARCHITECTURE

Select and define N-tier architectures suitable for scalable innovation.

Introducing the Running Example: JHUB Connect

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

The "Worked Example" Approach

Throughout this training, we will use **JHUB Connect**—a Student Innovation Project Management System—as our continuous case study. By applying every design principle to this real-world scenario, we bridge the gap between abstract theory and deployable software engineering.



Project Registration

Standardised capture of innovation ideas, team members, and developmental stages for JKUAT students.



Mentorship Hub

Automated scheduling and feedback loops between student innovators and industry-expert mentors.



Progress Tracking

Weekly milestone updates and documentation management to ensure project accountability and growth.



Investor Showcase

A public-facing portal for partners and investors to browse approved projects and support innovation.

TARGET USERS:

Students

Mentors

JHUB Admins

Global Investors

Understanding the System Design Document

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

What is an SDD?

A **System Design Document (SDD)** is a formal technical blueprint that translates abstract requirements into a concrete, buildable plan. It defines the technical **HOW** based on the functional **WHAT**.



Single Source of Truth

It provides a unified reference point that ensures all technical contributors stay aligned throughout the development lifecycle.

 Developers

 DBAs

 QA Testers

 UI/UX Designers

The Cost of Skipping Design

Without a formal SDD, JHUB Africa projects often face critical failures during the build phase:

- ❌ **Inconsistent Implementation:** Team members build features that don't talk to each other.
- ❌ **Chaotic Databases:** Redundant data structures that lead to corruption and poor performance.
- ❌ **Expensive Rework:** Fixing architectural flaws late in the cycle costs 10x more than upfront design.
- ❌ **Stakeholder Disconnect:** Failure to deliver the specific requirements promised in the SRS.

Professional Maturity

For JHUB students, the SDD demonstrates **technical readiness** to investors and partners. It proves you understand not just *what* to build, but *how* to build it reliably in a resource-constrained environment.

| Why System Design Matters at JHUB Africa

JHUB CONNECT: FROM REQUIREMENTS TO REALITY



Resource Efficiency

In resource-constrained environments, every hour counts. System design **prevents expensive rework** by identifying logical bottlenecks before a single line of code is written.



Investor Readiness

Global partners and investors look for structured thinking. A complete SDD demonstrates **technical maturity**—proving your project is a scalable product, not just a prototype.



Engineering Excellence

Proper design ensures long-term sustainability. It prevents **"architectural drift"** and conflicting interfaces, allowing the system to be maintained by evolving team members at JHUB.



Core Principle: "A great idea without a great design is just a wish. Design turns your vision into a buildable blueprint."

| Requirements Document vs System Design Document

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

THE INPUT

Requirements (SRS)

Focus: The "WHAT"

PRIMARY AUDIENCE

👤 Stakeholders, clients, and end-users

PRIMARY LANGUAGE

🗣️ Plain English, **User Stories**, and high-level use cases

CREATED AT

🕒 The inception phase, **before** design begins

CORE CONTENT

📋 System features, user roles, and business constraints



THE OUTPUT

System Design (SDD)

Focus: The "HOW"

PRIMARY AUDIENCE

👩‍💻 Developers, UI/UX designers, and DBAs

PRIMARY LANGUAGE

🔧 Technical: **Diagrams**, schemas, and wireframes

CREATED AT

✅ The planning phase, **after** requirements are approved

CORE CONTENT

🏗️ Architecture, DB schemas, UI mockups, and APIs

| The Four Pillars of System Design

JHUB CONNECT: FROM REQUIREMENTS TO REALITY



User Interface Design

Designing the structural blueprint of user interactions, information architecture, and screen flows.



Database Design

Defining how data is structured, stored, and related through entities, schemas, and integrity rules.



System Architecture Design

Determining how software components communicate, data flows, and where the system is deployed.



Non-Functional Requirements

Architecting for quality attributes: security, performance, scalability, and 99.5% system uptime.

Reading the Requirements Document

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

The Foundation

The **Software Requirements Specification (SRS)** defines the "What". It is the vital input that prevents architectural chaos.

"You cannot design what you have not understood."

For JHUB Connect, the SRS provides the raw material—from student registration to mentor scheduling—that we must convert into a buildable plan.

INPUT  SRS → DESIGN

Key Elements to Extract



User Roles / Actors

Identify who interacts with the system. Each role dictates specific user flows and access permissions.



Functional Requirements

List every feature the system must perform. These map directly to UI components and DB logic.



Non-Functional (NFR)

Performance, security, and scalability targets. These define the infrastructure and architecture choices.



Constraints & Assumptions

Tech stack choices, budget limits, or environment factors (e.g., mobile-first for 4G connectivity).

DESIGNER'S NOTE

Always cross-reference individual features with the overall project scope to ensure your design doesn't suffer from "Scope Creep".

Key Elements to Extract for Design

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

Before drafting a single line of architecture, you must "mine" the Requirements Document (SRS) for these five critical pillars. They form the foundation of your System Design Document (SDD).



USER ROLES

Actors

Who interacts with the system?

Permissions

What can each role access?

Example: Student vs Mentor



FUNCTIONAL

Features

Specific tasks the system performs.

Workflows

The logic flow of operations.

Example: Project Registration



NON-FUNCTIONAL

Quality

Performance, security, and scale.

Metrics

Measurable targets (e.g. 2s load).

Example: 99.5% Uptime



CONSTRAINTS

Limitations

Tech stack, budget, or timelines.

Environment

Infrastructure or browser support.

Example: React/Node Stack



ASSUMPTIONS

Context

What is taken for granted?

Dependencies

External systems or user skills.

Example: 4G Mobile Access

JHUB Connect: Roles & Functional Requirements

JHUB CONNECT: FROM REQUIREMENTS TO REALITY



Student

Primary innovator; registers projects, submits weekly updates, and requests mentorship.



Mentor

Subject expert; reviews student progress, provides feedback, and manages sessions.



Admin

System manager; oversees project lifecycles, assigns mentors, and generates reports.



Investor / Visitor

External partner; browses the public showcase of approved innovation projects.



Core Functional Requirements (FR)

FR1

Project Registration

Register innovations with title, description, team members, and category.

FR2

Progress Tracking

Submit weekly updates with milestone status and supporting attachments.

FR3

Mentorship Management

Request sessions, approve/decline requests, and document session notes.

FR4

Admin Approval Workflow

Administrative review of project submissions and mentor assignments.

FR5

Innovation Showcase

Public-facing directory filtered by category, stage, and location.

FR6

Automated Notifications

Email alerts for project approvals, session confirmations, and deadlines.

FR7

Reporting & Export

Generation of CSV/PDF reports on project health and mentor engagement.

JHUB Connect: Non-Functional Requirements

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

Non-Functional Requirements (NFRs) define the **quality attributes** of the system. While functional requirements describe *what* the system does, NFRs describe *how well* it performs its duties.



Performance & Scale

- ✓ **Latency:** Page load time under 2 seconds on a 4G mobile connection.
- ✓ **Concurrency:** Support 500 simultaneous users during peak events.

 Crucial for Demo Day events at JHUB Africa.



Security & Privacy

- ✓ **Encryption:** Data encrypted at rest (AES-256) and in transit (TLS 1.3).
- ✓ **Integrity:** Comprehensive audit logs for all project approvals.

 Protects student IP and sensitive mentor feedback.



Accessibility & UX

- ✓ **Responsive:** Full functionality on mobile browsers (375px+ width).
- ✓ **Contrast:** Minimum 4.5:1 ratio for all text elements.

 Ensures inclusivity for students on diverse hardware.



Availability

- ✓ **Uptime:** Target 99.5% uptime during academic semesters.
- ✓ **Recovery:** Database backups every 24 hours with <4h RTO.

SLA: 99.5%

Introduction to User Interface Design

JHUB CONNECT: FROM REQUIREMENTS TO REALITY



UI Design in the system design context is **not** the final visual polish. It is the **structural specification** of how users interact with the system.



System Design UI (The Blueprint)

- ✓ Information Architecture & hierarchy
- ✓ User flows and logical navigation
- ♥ Wireframes showing content placement
- 🏠 Input validation & component states



Visual Design (The Polish)

- ✗ Colour palettes and branding
- ✗ Typography styles and font-weights
- ✗ Shadows, gradients, and icons
- ✗ Micro-interactions and animations

THE SDD UI ROADMAP



Information Architecture

What screens exist and how are they organised?



User Flow Diagrams

What sequence of actions completes a task?



Wireframes

The schematic layout and content placement.



Component Specs

Rules, states, and validation for every element.

The Ultimate Goal: Every developer must know exactly what screen to build, every interaction to handle, and every state to display—**without ambiguity**.

Step 1: Information Architecture

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

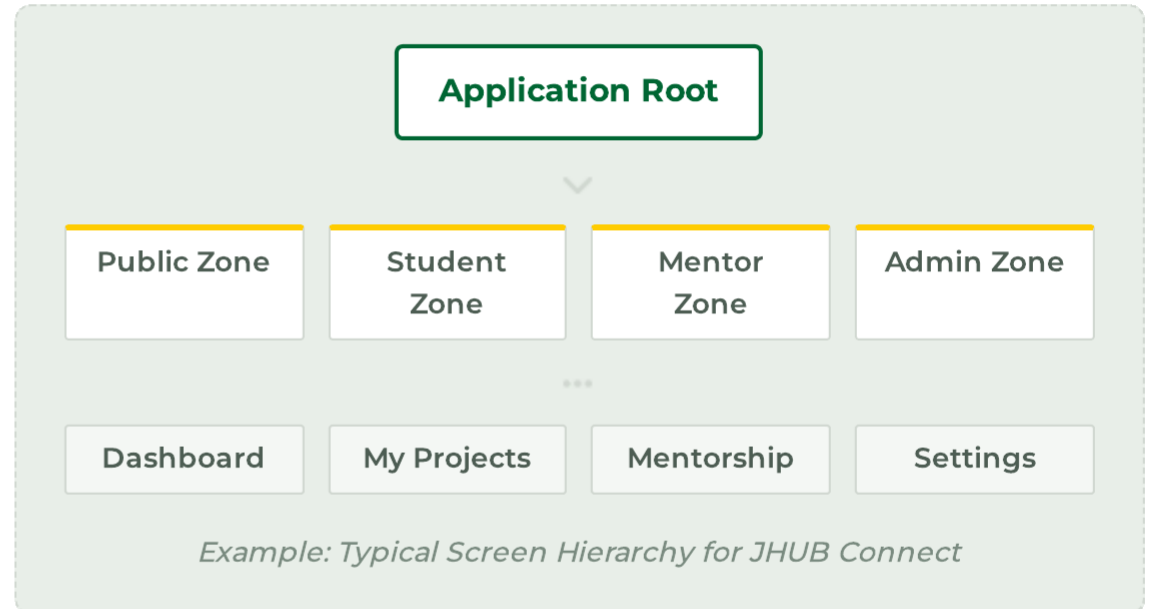
The Structural Blueprint

Information Architecture (IA) is the structural design of shared information environments. It organises content and functionality so users can find what they need intuitively.

- ▮ **Visual Inventory:** Defines every screen and module that must be built.
- ▮ **User Navigation:** Establishes the logical paths between different system zones.
- ▮ **Hierarchy:** Groups related features (e.g., Student vs Mentor tools).
- ▮ **Scalability:** Ensures new features have a logical place to live in the future.

 IA is about "How users find", not "How it looks".

Mapping the Hierarchy

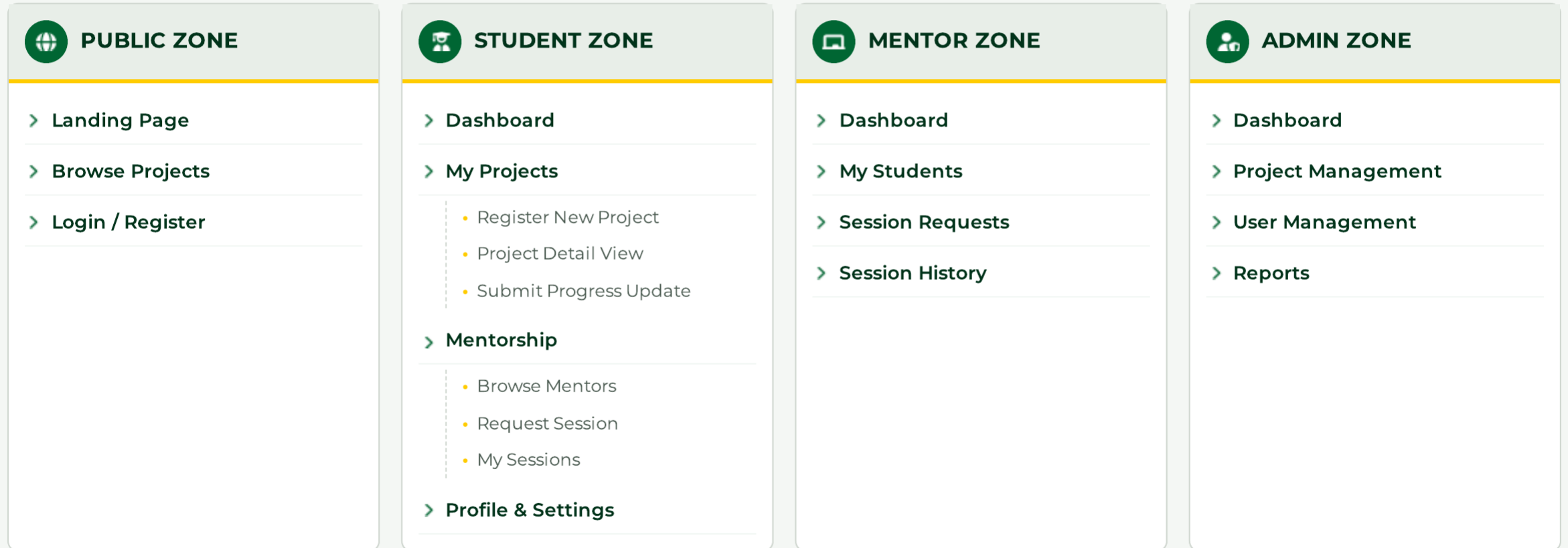


In system design, we represent this as a **Site Map** or **Screen Hierarchy Diagram**. It bridges the gap between the functional requirements and the final user interface.

JHUB Connect: Information Architecture Map

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

JHUB CONNECT PLATFORM



 This Information Architecture defines the site map and hierarchy used to guide technical wireframing and navigation logic.

Step 2: User Flow Diagrams

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

🧭 What is a User Flow?

A User Flow is a visual map showing the sequential path a user takes through the system to complete a specific goal. It bridges Information Architecture and final UI design.

🎯 The Design Goal

Ensure every developer knows exactly:

- Which screen to build next
- Every interaction to handle
- Every system state to display

STANDARD FLOW COMPONENTS



Entry Point

Where the task begins



Screen/Page

A specific UI view



User Action

Click, tap, or input



System Decision

Logic/Validation step



Integration

API call or DB update



Outcome

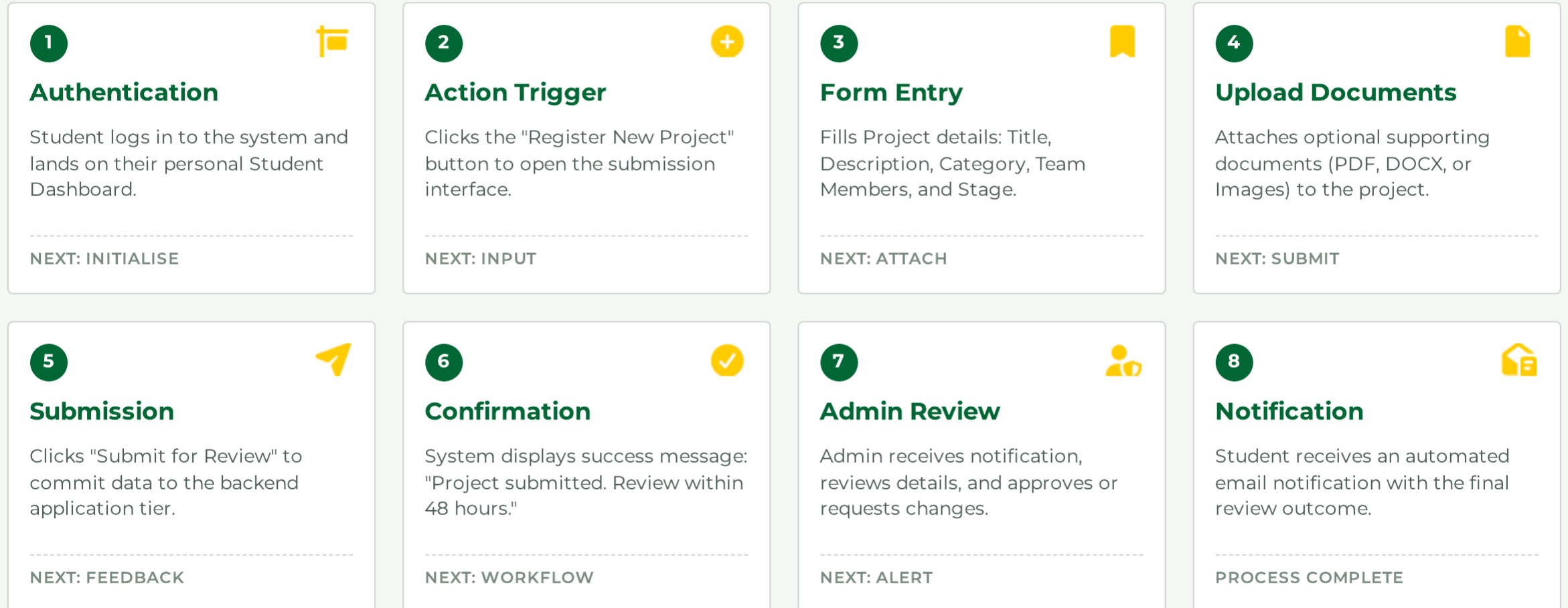
Success or Exit state



Pro Tip: Create one flow per core use case. Start with the "Happy Path" before mapping errors.

JHUB Connect: Project Registration Flow

JHUB CONNECT: FROM REQUIREMENTS TO REALITY



Design Note: This user flow maps the logical path from the *Requirements Document (SRS)* to the *User Interface (UI)*. Each step represents a state transition that must be handled by the UI components and the Backend API.

JHUB Connect: Mentorship Request Flow

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

PHASE 1: STUDENT INITIATION

STEP 01



Discovery & Filter

Student accesses the "Browse Mentors" view to find expertise matching their project category.

- Filter by AgriTech, HealthTech, etc.
- View mentor ratings & availability

STEP 02



Request Submission

Student clicks "Request Session" and completes the structured booking form.

- Input: Proposed Date & Time
- Input: Specific questions for Mentor

STEP 03



System Dispatch

Backend validates availability and pushes real-time notifications to the mentor.

- Entry created in database (Pending)
- Email & Dashboard alert triggered

STEP 04



Mentor Evaluation

Mentor reviews the student's project context and the proposed session agenda.

- Option A: Approve as requested
- Option B: Suggest alternative time

STEP 05



Student Feedback

Student receives the outcome. If an alternative was proposed, the student must confirm.

- Real-time status update in UI
- Final confirmation handshake

STEP 06



Finalisation & Sync

System locks the session, generates a meeting link, and updates both calendars.

- Calendar event (.ics) generated
- "My Sessions" dashboard updated

PHASE 2: MENTOR RESPONSE & SYSTEM FINALISATION

Step 3: Wireframing the System

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

DEFINITION



Low-Fidelity Schematics

Wireframes are skeletal sketches of each screen that serve as the technical blueprint for the interface. They are strictly **black and white** to prevent distractions from visual polish.

- ✓ Focuses entirely on structural logic and information flow.
- ✓ Bridges the gap between abstract requirements and visual design.
- ✓ Facilitates rapid iteration before any code is written.

STRATEGY



Function Over Aesthetics

By removing colour and images, the design team can focus on the core usability of the JHUB Connect platform.

- 🟡 **Layout:** Establishing spatial hierarchy and grid systems.
- 🟡 **Content:** Prioritising information based on user roles.
- 🟡 **Navigation:** Testing the logic of the Information Architecture.

COMPONENTS



What to Define

Every wireframe must clearly specify the functional zones of the interface without ambiguity for the developer.

- ➔ **Navigation Elements:** Sidebars, tabs, and breadcrumbs.
- ➔ **Interactive Zones:** Buttons, dropdowns, and input fields.
- ➔ **Visual Hierarchy:** Using size and weight to guide the eye.

Figma Balsamiq Sketch

BEHAVIOR



State Specifications

A professional SDD includes wireframes for all possible system states to ensure a seamless student experience.

- ✓ **Empty States:** "No projects registered yet."
- ✓ **Error States:** Validation messages and failed uploads.
- ✓ **Loading States:** Spinners and skeletons for slow networks.
- ✓ **Success States:** Confirmation banners after submission.

JHUB Connect: Student Dashboard Wireframe

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

 JHUB Connect

 Dashboard

 My Projects

 Mentorship

 Profile

[Navigation Sidebar]

Welcome back, Jane Wanjiru



MY PROJECTS

3

Total registered

SESSIONS

2 Due

Upcoming mentorship

Recent Activity

 Project "AquaSensor" approved by Admin
2 hours ago

 Mentor session confirmed: Friday 3:00 PM
1 day ago

 Progress update due for "SmartFarm"
Due tomorrow

+ Register New Project

Step 4: UI Component Specifications

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

Field Name	Type	Req.	Validation Rules	Max Len
Project Title	Text Input	YES	5–100 chars; alphanumeric; no special symbols allowed.	100
Description	Textarea	YES	50–1000 chars; must provide context of innovation.	1000
Category	Dropdown	YES	Select: AgriTech, HealthTech, FinTech, EdTech, CleanTech, Other.	—
Team Members	Multi-select	NO	Must be registered JKUAT students; maximum 10 members.	10
Stage	Radio Button	YES	Must select one: Idea, Prototype, Pilot, or Ready to Scale.	—
Supporting Docs	File Upload	NO	PDF/DOCX/IMG; Max 5MB per file; Max 3 files total.	3

 **Component States**

 **Default (Empty)**
Initial state with placeholders and labels visible.

 **In-Progress (Active)**
User is typing; real-time field highlighting enabled.

 **Validation Error**
Field highlighted red with context-specific error message.

 **Loading / Submitting**
Buttons disabled to prevent duplicate submissions.

 **Success State**
Confirmation message displayed; form cleared or redirected.

Key Guideline: Detailed specs ensure developers build exactly what is designed, preventing logic errors and inconsistent UI behaviour across the JHUB Connect platform.

UI Design Principles for JHUB Africa

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

01



Mobile-First Design PRIORITY

Design for the smallest screen (375px) first, then scale up. Most JHUB Africa users rely on mobile browsers for project tracking and engagement.

02



Accessibility & Inclusivity

Ensure a minimum contrast ratio of 4.5:1 for text. Provide visible labels for all form fields and maintain full keyboard navigability for accessibility.

03



Visual Consistency

Use a defined component library across all zones. Standardise button styles, typography scales, and grid systems to build user trust and speed up development.

04



Progressive Disclosure

Show only what the user needs at each step. Prevent cognitive overload by hiding complex options or advanced data until the user explicitly requests them.

| Introduction to Database Design

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

Definition

Database design is the process of defining the structure in which data will be stored, organised, and accessed in the system.

"It bridges the gap between abstract requirements and persistent reality."

A well-designed database is the backbone of any JHUB Africa project, ensuring that information is reliable, secure, and ready for future growth.

Data Integrity

Ensuring data is accurate, consistent, and follows business rules (e.g., a student must have a valid JKUAT email).

Reduce Redundancy

Eliminating duplicate data to prevent inconsistencies and save storage space through effective normalisation.

Query Performance

Designing structures that allow developers to retrieve information rapidly, even as the project count grows.

Scalability

Creating a flexible schema that can accommodate new features (like investor tracking) without breaking existing logic.

PRIMARY DELIVERABLES



ERD Diagram

Visual map of entities & relations



Table Schemas

Detailed field definitions & types



SQL Blueprints

DDL scripts for implementation

Identifying Entities and Attributes

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

The Noun Extraction Method

- **Identify Entities:** Scan the Requirements Document for nouns that represent real-world objects or concepts needing *persistence* (storage).
- **Filter Candidates:** Not every noun is an entity. Avoid transient objects (e.g., "button") or system outputs (e.g., "PDF export").

Defining Attributes

- **Identify Properties:** For each entity, determine what data fields are required to describe it (e.g., "Title", "Date").
- **Primary Keys:** Assign a unique identifier (ID) to every entity to ensure record uniqueness.

JHUB CONNECT: EXTRACTION LAB

Requirement Noun	Database Entity	Core Attributes
"The Student registers their profile..."	User	user_id email role phone
"Submit a new innovation Project ..."	Project	project_id title desc stage
"Request a Mentorship Session ..."	MentorshipSession	session_id proposed_at status
"Submit weekly Progress Updates ..."	ProjectUpdate	update_id content milestone
"Filter projects by Category ..."	Category	category_id category_name

**Note: Nouns like "Student", "Mentor", and "Admin" are often collapsed into a single "User" entity with a "role" attribute to maintain a clean schema.*

JHUB Connect: Core Entities and Attributes

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

 **User**

Attribute	Type & Constraint
user_id	PK <code>UUID</code>
email	<code>VARCHAR(100)</code> UNIQUE
first_name	<code>VARCHAR(50)</code>
last_name	<code>VARCHAR(50)</code>
role	<code>ENUM</code> (std men adm)
institution	<code>VARCHAR(100)</code>
created_at	<code>TIMESTAMP</code>

 **Project**

Attribute	Type & Constraint
project_id	PK <code>UUID</code>
title	<code>VARCHAR(200)</code>
description	<code>TEXT</code>
stage	<code>ENUM</code> (idea proto pilot)
status	<code>ENUM</code> (pend appr rej)
student_id	FK <code>UUID</code>
mentor_id	FK <code>UUID</code> (NULL)



MentorshipSession

Attribute	Type & Constraint
session_id	PK <code>UUID</code>
student_id	FK <code>UUID</code>
mentor_id	FK <code>UUID</code>
project_id	FK <code>UUID</code>
proposed_at	<code>DATETIME</code>
status	<code>ENUM</code> (req conf comp)
topic	<code>VARCHAR(300)</code>

 Data Integrity: Primary Keys (PK) ensure uniqueness, while Foreign Keys (FK) establish relationships between entities for relational consistency.

Defining Entity Relationships

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

↔ One-to-One (1:1)

Each record in Table A relates to exactly one record in Table B. Often used to extend data or for security isolation.

SINGLE ASSOCIATION

👤 One-to-Many (1:N)

The most common relationship. A single record in the parent table relates to multiple records in the child table.

PARENT-CHILD LOGIC

👥 Many-to-Many (M:N)

Multiple records in A relate to multiple in B. Requires a "Junction Table" to resolve the connection.

JUNCTION REQUIRED

🔗 JHUB Connect Relationship Mapping

RELATIONSHIP	TYPE	IMPLEMENTATION LOGIC
User → Project	1:N	One student can register many projects; student_id is the Foreign Key (FK).
Project → Update	1:N	One project has many weekly progress updates; project_id is the FK.
Mentor → Session	1:N	One mentor manages many mentorship sessions via mentor_id FK.
Project ↔ User	M:N	Team members. Resolved via ProjectMember junction table.
User → Profile	1:1	User record extends to a specific MentorProfile for specialized data.

The Entity-Relationship Diagram (ERD)

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

Visual Blueprint

The ERD maps how data entities relate to one another. It is the definitive guide for database implementation and query design.

CROW'S FOOT NOTATION

|| Mandatory One

| Optional One

>| Mandatory Many

>0 Optional Many

Key Rules

Every table must have a **PK**
Relationships use **FKs**
Avoid many-to-many (resolve)

USERS



PK user_id	UUID
email	VARCHAR
role	ENUM

One-to-Many with Projects

CATEGORIES



PK category_id	INT
name	VARCHAR

Lookup for Projects

PROJECTS



PK project_id	UUID	FK student_id	UUID
title	VARCHAR	FK category_id	INT
stage	ENUM	FK mentor_id	UUID

Central junction point for updates and sessions

PROJECT_UPDATES



PK update_id	UUID
FK project_id	UUID

Many-to-One with Project

MENTORSHIP_SESSIONS



PK session_id	UUID
FK mentor_id	UUID

Links User and Project

Database Normalisation Principles

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

1NF First Normal Form

✓ One value per column; no repeating groups

Every column must hold a single, atomic value. No lists, comma-separated strings, or arrays stored in a single field. Each row must be uniquely identifiable.

✗ **BAD:** `team_members` = "Jane, John, Peter" — multiple values in one column

✓ **GOOD:** Separate `ProjectMember` table — one row per member, each with its own `user_id`

JHUB Connect: Team members stored in `ProjectMember` junction table, not as a list in `Project`.

2NF Second Normal Form

✓ No partial dependencies on a composite key

Must already satisfy 1NF. Every non-key column must depend on the *entire* primary key — not just part of it. Applies when the primary key is composite (multiple columns).

✗ **BAD:** In `ProjectMember(project_id, user_id)`, storing `student_name` depends only on `user_id`

✓ **GOOD:** `student_name` belongs in the `User` table; `ProjectMember` stores only `role_in_project`

JHUB Connect: `ProjectMember` PK is `(project_id, user_id)` — only `role_in_project` and `joined_at` stored here.

3NF Third Normal Form

✓ No transitive dependencies between non-key columns

Must already satisfy 2NF. No non-key column should depend on another non-key column. Every attribute must depend directly on the primary key — nothing else.

✗ **BAD:** `Project` stores both `category_id` and `category_name` — name depends on `category_id`, not `project_id`

✓ **GOOD:** Separate `Category` table holds the name; `Project` stores only `category_id` FK

JHUB Connect: `Category(category_id, name)` is a standalone table — no duplication across project rows.

 **Golden Rule:** Normalise to **3NF** for all JHUB Africa projects — eliminates duplication, prevents update anomalies, and keeps your data trustworthy as the system scales.

Indexing Strategy for Performance

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

⚡ WHAT IS A DATABASE INDEX?

An index is a data structure that allows the database engine to locate rows **without scanning every row** in a table. Think of it as the index at the back of a textbook — instead of reading every page, you jump directly to the entry you need. Index the columns you *search*, *filter*, and *join* on.

✅ INDEX COLUMNS THAT ARE...

- Frequently used in **WHERE** clauses (filtering and search)
- Used in **JOIN** conditions between tables (foreign keys)
- Used in **ORDER BY** or **GROUP BY** clauses
- Columns enforcing **uniqueness** (email, username)

🗪 JHUB CONNECT — RECOMMENDED INDEXES

TABLE	COLUMN(S)	TYPE	REASON
users	email	UNIQUE	Login lookup — called on every authentication
projects	student_id	STANDARD	"My Projects" query — filtered by student
projects	status	STANDARD	Admin dashboard — filter by pending/approved
projects	category_id	STANDARD	Investor browse — filter by AgriTech, FinTech...
mentorship_sessions	mentor_id, student_id	COMPOSITE	Dashboard session queries — both FK lookups

⚖️ THE INDEXING TRADE-OFF

↑ BENEFIT

Dramatically faster **SELECT** queries, especially on large tables with thousands of rows

↓ COST

Slightly slower **INSERT / UPDATE / DELETE** — the index must be updated on every write

💡 *Read-heavy tables (projects, users) benefit most. Avoid over-indexing write-heavy log tables.*

🔗 SQL — CREATING INDEXES IN POSTGRESQL

```
-- Unique index: fast login lookups
CREATE UNIQUE INDEX idx_users_email
ON users(email);

-- Standard index: "My Projects" query
CREATE INDEX idx_projects_student
ON projects(student_id);

-- Composite: session dashboard queries
CREATE INDEX idx_sessions_mentor_student
ON mentorship_sessions(mentor_id, student_id);
```

★ GOLDEN RULE

Index columns you **search, filter, and join on** — not every column. Measure query performance with **EXPLAIN ANALYSE** before and after adding an index.

🗪 INDEX TYPE REFERENCE

- **UNIQUE** — Enforces uniqueness + speeds lookup (e.g., email)
- **STANDARD** — Speeds filtering on a single FK or status column
- **COMPOSITE** — Covers queries that filter on two columns together

Introduction to System Architecture

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

What is Architecture?

System architecture defines the high-level structure of the software system—the major components, how they communicate, and how they are deployed.

- ✔ Ensures technical consistency across the whole development team.
- ✔ Visualises the "Big Picture" before writing a single line of code.
- 🏠 Guides infrastructure choices and long-term scalability paths.
- 🏠 Identifies potential bottlenecks and security risks early.

 Architecture is the "Blueprint" that guides engineering decisions.

PATTERN	DESCRIPTION	BEST FOR
Monolithic	All components (UI, API, DB) in one deployable unit.	MVPs, Small teams
Microservices	Independent, decoupled services for each domain.	Large-scale systems
MVC	Separates Data (Model), UI (View), and Logic (Controller).	Standard Web apps
Client-Server	Clients send requests; server processes and responds.	Web/Mobile products
Layered (N-tier)	Separates Presentation, Application, and Data tiers.	JHUB Africa projects

JHUB Connect: 3-Tier Layered Architecture

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

CHOSEN PATTERN

A 3-Tier Layered Architecture for JHUB Connect

Presentation → Application → Data tiers give the team a clear separation of concerns, making the system straightforward to build, test, and deploy on affordable cloud infrastructure.

Right-sized for the team: Ideal for 3–6 developers working in parallel without stepping on each other.

Clear separation of concerns: Frontend, backend, and database evolve independently with well-defined interfaces.

Easy to test: Each tier can be unit-tested and integration-tested in isolation.

Affordable deployment: Each tier maps directly to a free or low-cost cloud host (Vercel, Railway, Supabase).

ARCHITECTURE TIER BREAKDOWN

1 Presentation Tier — Frontend What users see and interact with

React.js Responsive CSS Vercel / Netlify

- Renders dashboards, forms, and project showcases
- Manages local UI state; calls Application Tier via REST API (HTTPS / JSON)
- Mobile-first responsive design for all JHUB Africa users

↓ REST API · JSON OVER HTTPS

2 Application Tier — Backend Business logic, authentication, and API

Node.js + Express JWT Auth Railway / Render

- Auth & authorisation, project approval workflow, session scheduling
- Server-side validation, email notifications, file upload handling
- Exposes RESTful endpoints: *POST/api/projects*, *GET/api/sessions*

↓ SQL QUERIES · S3 / CLOUDINARY API

3 Data Tier — Database & Storage Persistence, caching, and file storage

PostgreSQL Redis Cache AWS S3 / Cloudinary

- PostgreSQL stores all relational data: users, projects, sessions, notifications
- Redis caches session tokens and high-frequency dashboard queries
- S3 / Cloudinary handles uploaded documents, images, and attachments

API Design and Security Principles

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

RESTful Conventions

- **Statelessness:** No client context stored on server; every request contains all info needed.
- **Resource-Oriented:** Use nouns for endpoints, not verbs.
- **Standard Methods:** Use GET (read), POST (create), PUT (update), DELETE (remove).

EXAMPLE ENDPOINT

```
GET /api/projects?category=agritech
```

```
POST /api/sessions/request
```

JWT Authentication

- **Token Issuance:** Server signs a payload (UID, role) upon valid login credentials.
- **Bearer Scheme:** Client sends token in the standard Authorization HTTP header.
- **Verification:** Server validates signature without querying the database every time.

HEADER STRUCTURE

```
Authorization: Bearer <jwt_token>
```

RBAC Authorisation

- **STUDENT** Can register projects and request mentorship.
- **MENTOR** Can view assigned students and confirm sessions.
- **ADMIN** Can approve projects and export system-wide reports.

MIDDLEWARE CHECK

```
if (user.role !== 'admin') return 403;
```

Security Best Practices



HTTPS Everywhere



Input Sanitisation



Error Masking



Rate Limiting

Recommended Technology Stack

JHUB CONNECT: FROM REQUIREMENTS TO REALITY



Frontend Tier

React.js & Tailwind CSS

Component-based UI development with utility-first responsive styling.

Vercel / Netlify

Automated deployment pipelines and global CDN for static hosting.

RATIONALE

Fastest time-to-market for modern web apps with massive community support and zero-cost entry tiers.



Application Tier

Node.js & Express.js

Scalable, non-blocking I/O runtime for building RESTful API services.

Render / Railway

Developer-friendly cloud hosting for backend services with easy scaling.

RATIONALE

Unified JavaScript ecosystem (Frontend + Backend) reduces context switching and simplifies development.



Data & Auth Tier

Supabase (PostgreSQL)

Managed relational database, built-in Auth (JWT), and real-time listeners.

Cloudinary

Optimised cloud storage and management for project media and documents.

RATIONALE

"Backend-as-a-Service" approach eliminates complex infrastructure management, allowing focus on core features.



The JHUB Advantage: This stack leverages **Managed Services** and **Modern Frameworks** to ensure high performance, security, and scalability while remaining entirely within the **Free Tier** for typical innovation prototypes.

The Complete SDD Template

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

01 FOUNDATIONS

- Project Scope & Objectives
- Target Audience & User Personas
- Technology Stack Rationale
- Development Constraints & Assumptions

03 DATABASE DESIGN

- Entity-Relationship Diagram (ERD)
- Data Dictionary (Field Definitions)
- Normalization Strategy (1NF to 3NF)
- Indexing & Query Optimization

05 QUALITY & NFRS

- Auth Strategy (JWT / OAuth2)
- Role-Based Access Control (RBAC)
- Performance & Uptime Targets
- Security (Encryption at Rest/Transit)

02 UI DESIGN

- Information Architecture (Sitemap)
- Critical User Flows
- High-Fidelity Wireframes
- Component Library & Style Guide

04 ARCHITECTURE

- System Context Diagram
- Layered Architectural Blueprint
- RESTful API Endpoint Definitions
- Third-Party Service Integrations

06 OPERATIONS

- CI/CD Pipeline Overview
- Environment Setup (Dev/Staging/Prod)
- Backup & Disaster Recovery Plan
- Post-Launch Maintenance Schedule



Dr. Lawrence's Pro-Tip: Treat this document as a "living" blueprint. Update it as your design evolves, but ensure every change is approved by the team before implementation begins.

Common Design Mistakes to Avoid

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

Neglecting Non-Functional Requirements

Prioritising features (FRs) while ignoring security, latency, and concurrent user capacity until the deployment phase.

 **CRITICAL**

System Crash During Demo Day

Poor Database Normalisation

Storing redundant data or failing to reach 3NF, leading to data anomalies and difficult maintenance as the project grows.

 **SEVERE**

Data Corruption & Integrity Loss

Hard-Coding Environment Secrets

Embedding API keys, database credentials, or server URLs directly in the source code instead of using environment variables.

 **HIGH RISK**

Security Breach & Credential Leak

Monolithic UI Components

Building massive, single-file screens without reusable atoms or molecules, causing high development debt and inconsistency.

 **MODERATE**

Increased Maintenance Time

Opaque Error & Empty States

Designing only the "happy path" and neglecting how the UI responds to server timeouts, validation errors, or empty datasets.

 **HIGH**

Poor User Retention & Frustration

Summary and Next Steps

JHUB CONNECT: FROM REQUIREMENTS TO REALITY

Design Workflow Checklist



Requirement Extraction

Transform abstract SRS user stories into concrete functional specifications and data entities.



UI/UX Blueprinting

Finalise Information Architecture and high-fidelity wireframes for all core user journeys.



Data Architecture

Ensure 3NF normalisation, efficient indexing, and clear ERD relationships for scalability.



System Integrity

Validate the 3-Tier architecture against NFRs for security, performance, and uptime.

REALITY

"A great design document is the bridge that turns a student vision into an investor-ready innovation."

ACTION ITEMS FOR STUDENTS

- Download the JHUB Africa SDD Template
- Schedule a Design Review with your Mentor
- Finalise Tech Stack & Environment Setup

Dr. Lawrence Nderu | JHUB Africa, JKUAT